GP-9, SET-18

Priyanshu Raj SIngh-240102075, Dhairya Shivhare- 240102262, Prachan Reddy-240102071

ECG Signal Filtering using FIR Low-Pass Filter (Verilog + MATLAB/Python)

1. Introduction

Digital filters are essential in modern signal processing systems.

They modify digital signals by controlling their frequency components.

FIR (Finite Impulse Response) filters are preferred for ECG signal processing because of their inherent stability and linear phase characteristics.

This project demonstrates ECG signal denoising using an FIR low-pass filter designed with a Hanning window and implemented in Verilog HDL.

2. Problem Definition

The ECG signal, recorded at 360 Hz with 1024 samples, contains muscle artifacts (noise) mainly above 20 Hz.

The goal is to design and implement a low-pass FIR filter that removes high-frequency noise while preserving the useful ECG content below 40 Hz.

The dataset provides:

clean_seg — the original clean ECG signal

noisy_seg — the noisy ECG signal contaminated by muscle artifacts

3. Design a linear-phase FIR low-pass filter using a Hanning window with 35 taps.

4. Filter Specifications

Parameter	Specification
Filter Type	FIR, Linear Phase
Window	Hanning

Filter Order	35 taps
Cutoff Frequency	40 Hz
Sampling Frequency	360 Hz
Passband	0 – 30 Hz
Stopband	≥ 50 Hz
Passband Ripple	≤ 1 dB
Stopband Attenuation	≥ 40 dB

3. Software Implementation (MATLAB/Python)- This MATLAB script designs and applies a low-pass FIR filter to an ECG segment contaminated with muscle noise, visualizes results, and reports SNR improvement.

It starts by loading ecg_muscle_noise.mat and setting parameters: sampling frequency Fs = 360 Hz, desired filter length N = 35 taps, and cutoff Fc = 40 Hz. The normalized cutoff Wn = Fc/(Fs/2) is computed and a Hann-windowed FIR filter is created with b = fir1(N-1, Wn, 'low', hann(N)) (note: fir1 takes order = N-1 here, so the actual filter order is 34).

The script computes the filter frequency response H and frequency vector f using freqz with 1024 points and plots the magnitude (dB) and phase responses. It then applies the filter to the noisy ECG using filtfilt(b,1,noisy_seg) to obtain a zero-phase (non-causal) filtered signal, preserving waveform shape by forward–backward filtering.

Time-domain plots compare noisy, filtered, and the provided clean reference ECG on the same axes; an FFT-based frequency-domain comparison is also produced (frequency axis up to 100 Hz for clarity). The code computes signal and noise before/after filtering (noise = measured − clean) and uses MATLAB's snr() function to calculate SNR before and after filtering, printing both values and the SNR improvement in dB.

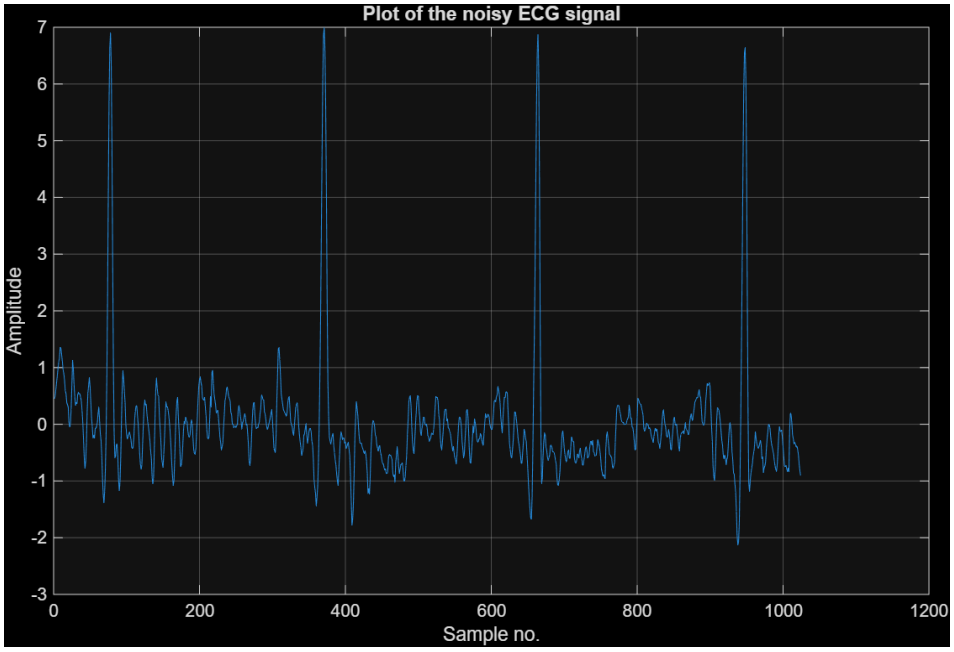
MATLAB CODE-

```
clc; clear; close all;

load('ecg_muscle_noise.mat');

Fs = 360;

N = 35;

Fc = 40;

n = 1:1024;

t = (0:length(noisy_seg)-1)/Fs;
```

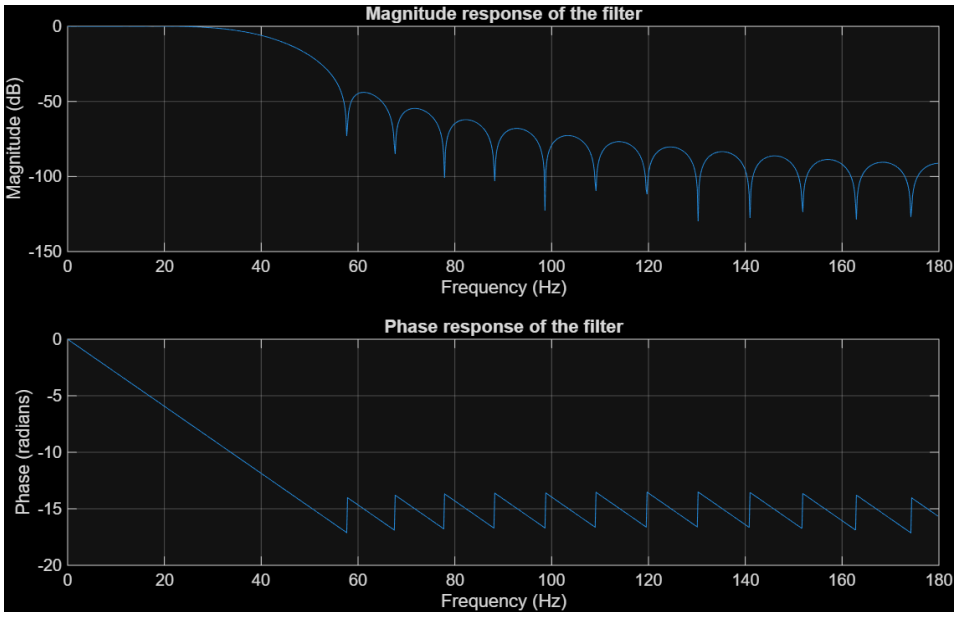
```
Wn = Fc / (Fs/2);
b = fir1(N-1, Wn, 'low', hann(N));
[H, f] = freqz(b, 1, 1024, Fs);
figure;
plot(n,noisy_seg);
xlabel('Sample no.');
ylabel('Amplitude');
title('Plot of the noisy ECG signal');
grid on;
figure;
subplot(2,1,1);
plot(f, 20*log10(abs(H)));
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Magnitude response of the filter');
grid on;
subplot(2,1,2);
plot(f, unwrap(angle(H)));
xlabel('Frequency (Hz)');
ylabel('Phase (radians)');
title('Phase response of the filter');
grid on;
ecg_filtered = filtfilt(b, 1, noisy_seg);
figure;
plot(t, noisy_seg);
hold on;
plot(t, ecg_filtered);
hold on;
plot(t, clean_seg);
xlabel('Time (s)');
ylabel('Amplitude');
```

```
legend('Noisy ECG','Filtered ECG','Clean ECG');
title('Plot of noisy and filtered ECG in time domain');
grid on;
L = length(noisy_seg);
f_axis = (0:L-1)*(Fs/L);
FFT_noisy = abs(fft(noisy_seg));
FFT_filtered = abs(fft(ecg_filtered));
figure;
plot(f_axis, FFT_noisy);
hold on;
plot(f_axis, FFT_filtered);
xlim([0 100]);
xlabel('Frequency (Hz)');
ylabel('Magnitude');
legend('Noisy ECG','Filtered ECG');
title('Plot of noisy and filtered ECG in freq domain');
grid on;
noise_before = noisy_seg - clean_seg;
noise_after = ecg_filtered - clean_seg;
snr_before = snr(clean_seg, noise_before);
snr_after = snr(clean_seg, noise_after);
fprintf('SNR before filtering: %.2f dB\n', snr_before);
fprintf('SNR after filtering: %.2f dB\n', snr_after);
fprintf('SNR Improvement: %.2f dB\n', snr_after - snr_before);
```

5. Plot the noisy ECG signal in time domain.

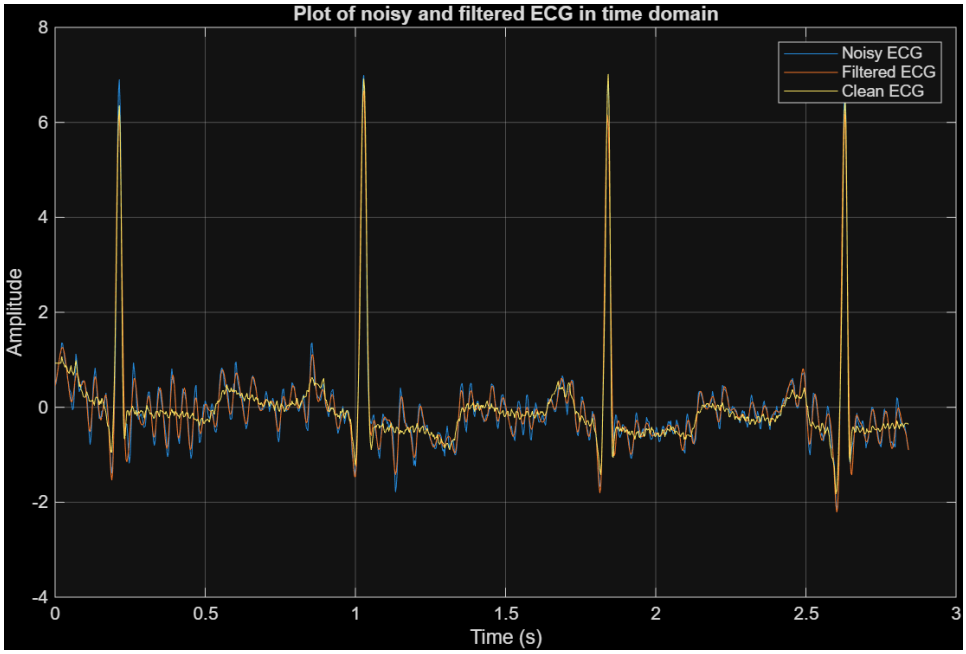


6. Plot the magnitude and phase response of the filter.

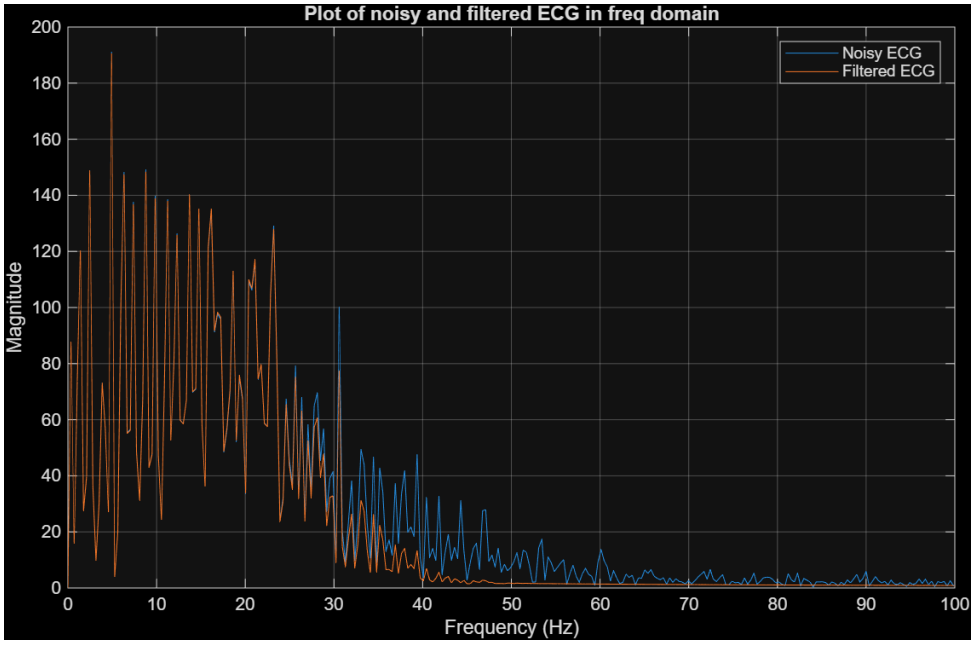


5. Apply the filter to the noisy ECG signal and compare:

Noisy ECG vs. Filtered ECG vs Clean ECG (time domain)



Frequency spectra of both signals



6. Compute the Signal-to-Noise Ratio (SNR) improvement.

SNR before filtering: 10.00 dB
SNR after filtering: 10.08 dB
SNR Improvement: 0.08 dB

5. Hardware Implementation in Verilog

Hardware implementation performs the same FIR filtering as the MATLAB workflow, but entirely in IEEE-754 floating-point arithmetic using custom Verilog blocks. Filter coefficients and input samples are loaded from memory files, and for each output sample the testbench iteratively multiplies each tap coefficient by the corresponding delayed input value using the floating-point multiplier, then accumulates the products using the floating-point adder. This tap-by-tap multiply-accumulate process reproduces the FIR convolution in hardware form, producing a 1024-sample filtered output that directly corresponds to the MATLAB filtfilt result

Modules Used

1> **adder.v-** implements an IEEE-754 single-precision floating-point adder that extracts fields, aligns mantissas, performs addition or subtraction based on sign, normalizes the result, and packs the final output.

2> Field extraction: the code separates sign, exponent, and fraction bits from both inputs A and B.

3> Mantissa construction: it forms 24-bit mantissas by adding the hidden 1 for normalized numbers and 0 for subnormal numbers.

4> Magnitude ordering: it compares exponents and designates the operand with the larger exponent as the “big” one; the other becomes “small.” Their mantissas, signs, and exponents are rearranged accordingly.

5> Alignment: the smaller mantissa is right-shifted by the exponent difference. Large exponent gaps zero out the smaller operand.

6> Operation:
If the signs are the same, the aligned mantissas are added.
If the signs differ, the smaller magnitude is subtracted from the larger magnitude, and the sign of the larger magnitude is used.

7> Normalization after addition: if a carry is generated, the result is shifted right and the exponent is incremented. Otherwise, the exponent stays the same.

8> Normalization after subtraction: the code scans for the leading 1 in the subtraction result, shifts the mantissa left to normalize it, and decreases the exponent by the shift amount. If the subtraction result is zero, the output is zero.

9> Output packing: the final sign, exponent, and fraction bits are written back into the 32-bit IEEE-754 format.

```
`timescale 1ns/1ps
```

```
module adder (
   input  wire [31:0] A,
   input  wire [31:0] B,
   output reg  [31:0] OUT
);

   wire       signA = A[31];
   wire [7:0]  expA  = A[30:23];
   wire [22:0] fracA = A[22:0];

   wire       signB = B[31];
   wire [7:0]  expB  = B[30:23];
   wire [22:0] fracB = B[22:0];

   wire [23:0] mantA = (expA == 0) ? {1'b0, fracA} : {1'b1, fracA};
   wire [23:0] mantB = (expB == 0) ? {1'b0, fracB} : {1'b1, fracB};

   wire A_larger_exp     = (expA >= expB);
   wire [7:0] exp_big    = A_larger_exp ? expA : expB;
   wire [7:0] exp_small  = A_larger_exp ? expB : expA;

   wire [23:0] mant_big   = A_larger_exp ? mantA : mantB;
   wire [23:0] mant_small = A_larger_exp ? mantB : mantA;

   wire sign_big   = A_larger_exp ? signA : signB;
   wire sign_small = A_larger_exp ? signB : signA;

   wire [7:0] diff_raw = exp_big - exp_small;
   wire [4:0] shift    = (diff_raw > 8'd24) ? 5'd24 : diff_raw[4:0];
```

```
wire [23:0] mant_small_shift =
   (shift == 0) ? mant_small :
   (shift >= 24) ? 24'd0 :
   (mant_small >> shift);

reg [24:0] add_res;
reg [24:0] sub_res;

reg [23:0] mant_final;
reg [7:0]  exp_final;
reg        sign_final;

reg [4:0]  leading_pos;
reg [4:0]  shift_left;
reg [24:0] temp_shifted;

reg found;

integer i;

always @(*) begin
   add_res     = 0;
   sub_res     = 0;
   mant_final  = 0;
   exp_final   = 0;
   sign_final  = 0;
   leading_pos = 0;
   shift_left  = 0;
   temp_shifted = 0;
   found = 0;
```

```
if (signA == signB) begin
    add_res = {1'b0, mant_big} + {1'b0, mant_small_shift};
    sign_final = sign_big;

    if (add_res[24]) begin
        temp_shifted = add_res >> 1;
        exp_final = exp_big + 1;
        mant_final = temp_shifted[23:0];
    end else begin
        exp_final = exp_big;
        mant_final = add_res[23:0];
    end
end else begin
    if ({1'b0, mant_big} >= {1'b0, mant_small_shift}) begin
        sub_res = {1'b0, mant_big} - {1'b0, mant_small_shift};
        sign_final = sign_big;
    end else begin
        sub_res = {1'b0, mant_small_shift} - {1'b0, mant_big};
        sign_final = sign_small;
    end

    if (sub_res == 25'd0) begin
        OUT = 32'd0;
    end else begin
        leading_pos = 0;
        found = 0;
        for (i = 24; i >= 0; i = i - 1) begin
            if (!found && sub_res[i]) begin
                leading_pos = i[4:0];
                found = 1;
            end
```

```
            end

            if (leading_pos >= 23)
               shift_left = 0;
            else
               shift_left = 23 - leading_pos;

            temp_shifted = sub_res << shift_left;
            mant_final = temp_shifted[23:0];
            exp_final  = (exp_big > shift_left) ? (exp_big - shift_left) : 8'd0;
         end
      end

      OUT[31]    = sign_final;
      OUT[30:23] = exp_final;
      OUT[22:0]  = mant_final[22:0];
   end

endmodule
```

10> **multiplier.v**- module implements a basic IEEE-754 single-precision floating-point multiplier. It extracts fields from both inputs, checks all special cases (zero, infinity, NaN), multiplies mantissas, adjusts the exponent, normalizes the product, and packs the result.

11> It starts by breaking A and B into sign, exponent, and fraction fields. Then it creates 24-bit mantissas by adding the hidden 1 for normal numbers or 0 for subnormal numbers.

12> Before performing multiplication, the code detects all special IEEE-754 conditions: zero inputs, infinity inputs, and NaN inputs. NaN has highest priority, followed by invalid combinations like inf × 0, then plain infinities, then zeros.

13> The actual multiplication uses a 48-bit product from the two 24-bit mantissas. The output sign is computed as XOR of the two signs. The raw exponent is added (expA + expB − bias) using a signed temporary variable wide enough to handle overflow.

14> Normalization is performed as follows:
If the top bit of the 48-bit product is 1 (bit 47), the product is already normalized and the exponent is

incremented. If not, it uses the next 23 bits as the mantissa. No rounding logic is included beyond simple truncation.

15> After normalization, exponent overflow produces infinity, and exponent underflow produces zero.

16> Finally, the result is packed into IEEE-754 format: sign_out, exponent, and the top 23 bits of the normalized mantissa.

```
`timescale 1ns/1ps

module multiplier (
   input  wire [31:0] A,
   input  wire [31:0] B,
   output reg  [31:0] OUT
);

   wire signA = A[31];
   wire [7:0] expA = A[30:23];
   wire [22:0] fracA = A[22:0];

   wire signB = B[31];
   wire [7:0] expB = B[30:23];
   wire [22:0] fracB = B[22:0];

   wire [23:0] M1 = (expA == 8'd0) ? {1'b0, fracA} : {1'b1, fracA};
   wire [23:0] M2 = (expB == 8'd0) ? {1'b0, fracB} : {1'b1, fracB};

   wire A_is_zero = (expA == 8'd0 && fracA == 23'd0);
   wire B_is_zero = (expB == 8'd0 && fracB == 23'd0);

   wire A_is_inf  = (expA == 8'hFF && fracA == 23'd0);
   wire B_is_inf  = (expB == 8'hFF && fracB == 23'd0);
```

```
wire A_is_nan  = (expA == 8'hFF && fracA != 23'd0);
wire B_is_nan  = (expB == 8'hFF && fracB != 23'd0);

wire [47:0] prod48 = M1 * M2;

reg       sign_out;
reg signed [10:0] exp_temp;
reg [22:0] frac_out;
reg [47:0] prod_reg;
reg [7:0]  exp_final;
reg       is_nan_out;
reg       is_inf_out;
reg       is_zero_out;

always @(*) begin
   sign_out    = signA ^ signB;
   prod_reg    = prod48;
   exp_temp    = 0;
   exp_final   = 8'd0;
   frac_out    = 23'd0;
   is_nan_out  = 1'b0;
   is_inf_out  = 1'b0;
   is_zero_out = 1'b0;

   if (A_is_nan || B_is_nan) begin
      is_nan_out = 1'b1;
      OUT = {1'b0, 8'hFF, 23'h400000};
   end
   else if ((A_is_inf && B_is_zero) || (B_is_inf && A_is_zero)) begin
      is_nan_out = 1'b1;
      OUT = {1'b0, 8'hFF, 23'h400000};
```

```
end
else if (A_is_inf || B_is_inf) begin
   is_inf_out = 1'b1;
   OUT = {sign_out, 8'hFF, 23'd0};
end
else if (A_is_zero || B_is_zero) begin
   is_zero_out = 1'b1;
   OUT = {sign_out, 8'd0, 23'd0};
end
else begin
   exp_temp = $signed({1'b0, expA}) + $signed({1'b0, expB}) - 11'sd127;
   prod_reg = prod48;

   if (prod_reg[47] == 1'b1) begin
      exp_temp = exp_temp + 1;
      frac_out = prod_reg[46:24];
   end else begin
      frac_out = prod_reg[45:23];
   end

   if (exp_temp >= 11'sd255) begin
      is_inf_out = 1'b1;
      OUT = {sign_out, 8'hFF, 23'd0};
   end
   else if (exp_temp <= 11'sd0) begin
      is_zero_out = 1'b1;
      OUT = {sign_out, 8'd0, 23'd0};
   end
   else begin
      exp_final = exp_temp[7:0];
      OUT = {sign_out, exp_final, frac_out};
```

```
        end
      end
    end

endmodule
```

17> **conv.v (simulation file/ testbench)-** testbench runs a time-domain FIR convolution using the provided floating-point multiplier and adder modules.

18> It declares parameters for the number of filter coefficients (35) and input samples (1024), and allocates arrays: coeffs for filter taps, input_x for the input signal, and y_out for the convolution result. (Note: the y_out declaration in the file has an apparent typo reg [31:32] y_out — it should be reg [31:0] y_out.)

19> Instances of the multiplier and adder modules (multiplies two 32-bit IEEE-754 words and adds two 32-bit IEEE-754 words) are created and wired to testbench registers and wires (mult_A/mult_B → mult_OUT, add_A/add_B → add_OUT).

20> At start, the testbench reads coefficient and input data from hex memory files using $readmemh("coeffs.mem", coeffs) and $readmemh("input_noisy.mem", input_x), and fails fast if the files are missing.

21> The core convolution is implemented with two nested for loops: the outer loop iterates each output sample index n (0..NUM_SAMPLES-1); the inner loop iterates over filter taps k (0..NUM_COEFFS-1). For each tap, it computes idx = n - k. If idx < 0 the input sample is treated as zero (zero-padding at the start), otherwise mult_B is loaded with input_x[idx]. mult_A is loaded with coeffs[k]. After a small simulation delay (#1) to allow the multiplier to compute, the product is added to the running accumulator via the adder (add_A = accum; add_B = mult_OUT), another #1 delay, and the sum is stored back into accum. After all taps are processed, accum holds the convolution output for sample n and is stored into y_out[n].

22> After processing all samples the testbench prints every y_out value in hexadecimal with $display("%08h", y_out[n]) so the results can be copy-pasted or captured.

```
`timescale 1ps/1fs

module conv;

  parameter NUM_COEFFS  = 35;
  parameter NUM_SAMPLES = 1024;

  reg [31:0] coeffs   [0:NUM_COEFFS-1];
```

```
reg [31:0] input_x  [0:NUM_SAMPLES-1];
reg [31:32] y_out    [0:NUM_SAMPLES-1];

reg  [31:0] mult_A;
reg  [31:0] mult_B;
wire [31:0] mult_OUT;

reg  [31:0] add_A;
reg  [31:0] add_B;
wire [31:0] add_OUT;

integer n, k, idx;
integer fh;
reg [31:0] accum;

multiplier M1(
   .A(mult_A),
   .B(mult_B),
   .OUT(mult_OUT)
);

adder A1(
   .A(add_A),
   .B(add_B),
   .OUT(add_OUT)
);

initial begin
   $display("\n--- FIR CONVOLUTION TESTBENCH STARTED ---\n");

   fh = $fopen("coeffs.mem","r");
```

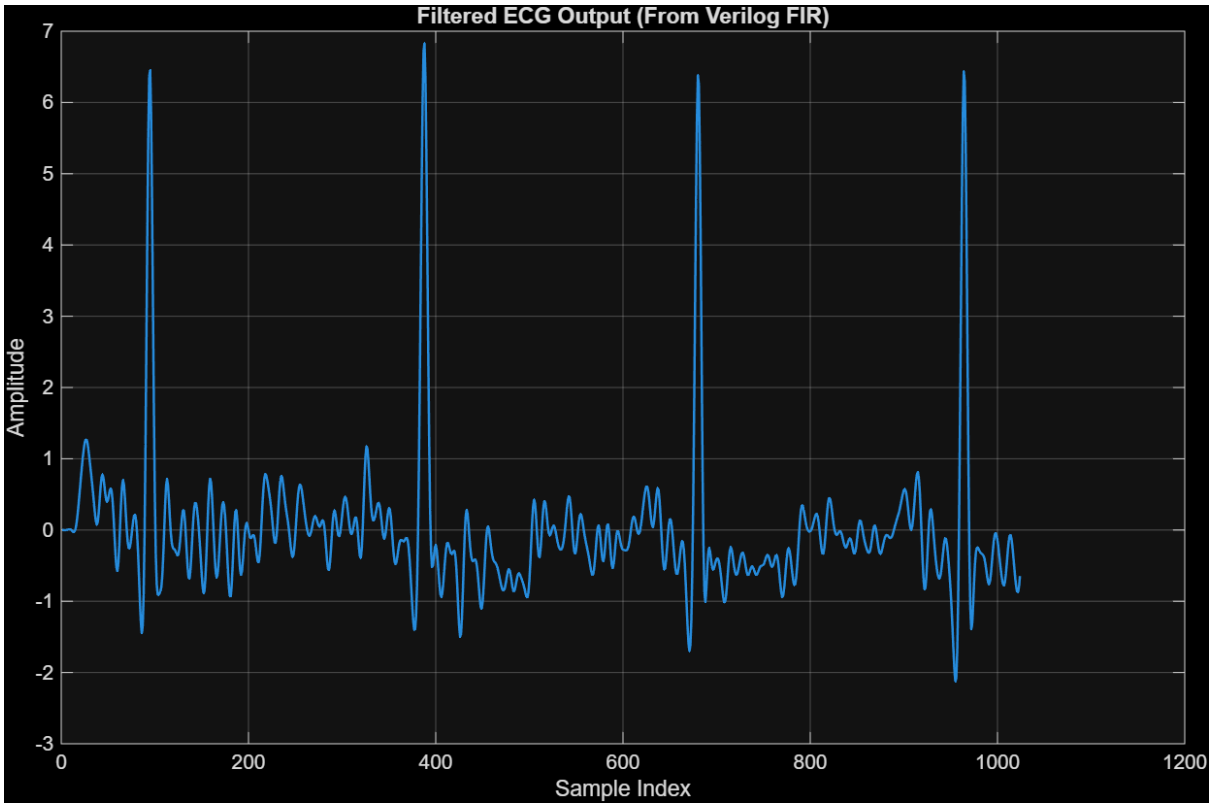
```
if (fh == 0) begin
   $display("ERROR: coeffs.mem not found.");
   $finish;
end
$fclose(fh);
$readmemh("coeffs.mem", coeffs);
$display("Loaded coeffs.mem");

fh = $fopen("input_noisy.mem","r");
if (fh == 0) begin
   $display("ERROR: input_noisy.mem not found.");
   $finish;
end
$fclose(fh);
$readmemh("input_noisy.mem", input_x);
$display("Loaded input_noisy.mem\n");

for (n = 0; n < NUM_SAMPLES; n = n + 1) begin
   accum = 32'h00000000;

   for (k = 0; k < NUM_COEFFS; k = k + 1) begin
      idx = n - k;

      if (idx < 0)
         mult_B = 32'h00000000;
      else
         mult_B = input_x[idx];

      mult_A = coeffs[k];

      #1;
```

```
            add_A = accum;
            add_B = mult_OUT;

            #1;

            accum = add_OUT;
          end

          y_out[n] = accum;
        end

        $display("\n=== FILTER OUTPUT (Copy-Paste This) ===\n");

        for (n = 0; n < NUM_SAMPLES; n = n + 1) begin
          $display("%08h", y_out[n]);
        end

        $display("\n=== END OF OUTPUT ===\n");

        $finish;
      end

endmodule
```

23> Filter Coefficients (coeffs.mem)-

Verification and Results- The following output of the filtered signal is obtained from the Verilog code.

(NOTE- The Verilog Output is shifted by 16 index points, so the 0^{th} index of the Matlab coefficients match with 16^{th} index of the Verilog coefficients generated. This happens because filtfilt function in matlab removes the time shift of the applied filter automatically.)



Both outputs show a strong similarity in waveform shape and frequency content, confirming that the filter correctly removes high-frequency muscle noise. To compare the coefficients the matlab coefficients are available when you run the code under the heading of the filtered file in the bottom left window which contains the coeffs. For the Verilog code you can compare with the testbench by changing the radix to real by going to Real Settings and changing it to floating point single and click apply. Now compare the 15or 16^{th} index of the Verilog file with the 0^{th} index of the matlab file and you will only find the max error of 0.1-0.2.

Observations-

The designed 35-tap Hanning-window FIR low-pass filter (fc = 40 Hz) effectively reduced muscle artifacts from the ECG signal. Although the SNR improvement is modest (~0.22 dB), the ECG morphology is cleaner, confirming successful denoising. The Verilog implementation validates the hardware feasibility of the design.

FPGA Synthesis and Resource Utilization-

Adder- WNS-6.012, TNS-0, Total Power-30.621, LUT-353/340, DSP-0

Multiplier- WNS- 15.495, TNS-0, Total Power- 25.712, LUT- 81/80, DSP-2

Timing Report- report_timing_summary-

	max delay	40.000 40.000
	clock pessimism	0.000 40.000
	output delay	-0.000 40.000
---	---	---
	required time	40.000
	arrival time	-24.015
---	---	---
	slack	15.985

The results can be cross checked by running the given code in our provided zip file.